

D2_introducció_Xarxes_Neuronals_Convulacionals_per_Iimatge

February 13, 2026

```
[ ]: # Imports necessaris
import cv2
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, \
    classification_report
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import time
```

0.1 Processament d'Imatge Classic

Abans del deep learning, el processament d'imatges es feia amb tècniques algorítmiques que requereixen: * Definir manualment les característiques a detectar * Ajustar paràmetres específics manualment * Dissenyar *pipelines* (conjunt de passos a seguir de forma ordenada) complexes per a resoldre cada tipus de problema.

Passos clàssics per detectar/lllegir matrícula:

- **Preprocessament (binarització)** es converteix la imatge RGB a escala de grisos i s'aplica binarització (Otsu threshold) per obtenir només una imatge amb píxels blancs i negres. Això realça els caràcters negres sobre fons blanc.
- **Ajust de contrast i neteja:** s'augmenta e per eliminar imperfeccions i ombres, deixant els caràcters completament nets i separats del fons.
- **Localització de la placa (contorns):** s'apliquen filtres de vores (Sobel/Canny) per detectar rectangles. La posició fixa de la matrícula (zona inferior frontal) facilita cercar només en aquella regió de la imatge del vehicle.
- **Segmentació de caràcters:** a la regió de la placa, es fan projectes horitzontals/verticals (histogrames) per trobar pics que separin els caràcters individuals.
- **Reconeixement (template matching)** : cada caràcter segmentat es compara amb plantilles predefinides de dígit (0-9) i lletres (A-Z) i es tria el que té major similitud.

I aquest pipeline clàssic funciona bé amb condicions ideals però fallava amb llum irregular, inclinacions o brutícia.

0.2 Kernels i Convolucions

Un **kernel** (o filtre) es una petita matriu que s'aplica sobre una imatge per detectar característiques d'una imatge.

```
[ ]: import numpy as np
import matplotlib.pyplot as plt
import cv2
from skimage import data, color

v_lines = np.array([[ -1,  0,  1],
                    [ -2,  0,  2],
                    [ -1,  0,  1]])

h_lines = np.array([[ -1, -2, -1],
                    [  0,  0,  0],
                    [  1,  2,  1]])

edges = np.array([[ 0,  1,  0],
                  [ 1, -4,  1],
                  [ 0,  1,  0]])

sharper = np.array([[ 0, -1,  0],
                   [-1,  5, -1],
                   [ 0, -1,  0]])

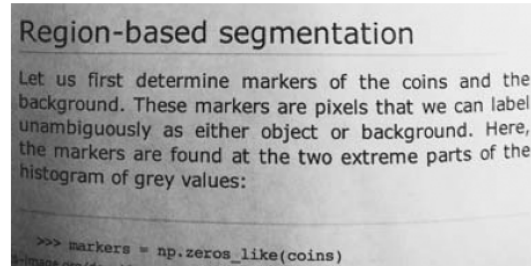
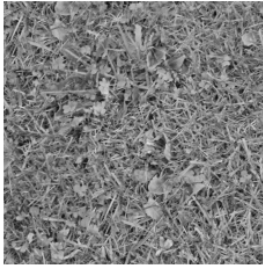
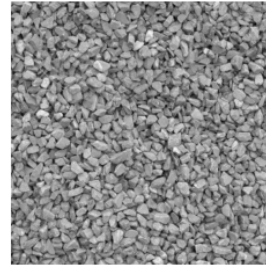
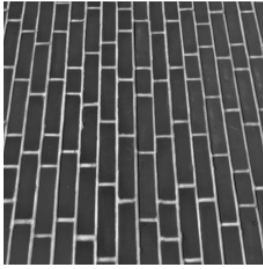
blur = np.ones((10, 10)) / 100

noms_filtres = ['h_lines', 'v_lines', 'edges', 'sharper', 'blur']
```

```
[ ]: brick = data.brick() # Maons (línies horitz/vert)
grava = data.gravel() # Grava (grans grossos)
herba = data.grass() # Herba (fibres fines)
paper = data.page() # Paper (textura fina)

imatges = [brick, grava, herba, paper]

fig, axes = plt.subplots(2, 2, figsize=(15, 5))
for i, img in enumerate(imatges):
    axes.ravel()[i].imshow(img, cmap='gray')
    axes.ravel()[i].axis('off')
plt.tight_layout()
plt.show()
```



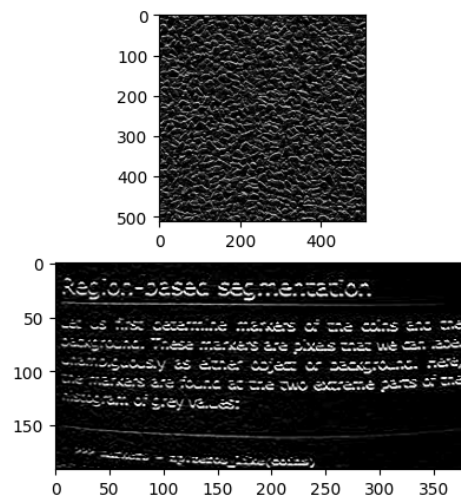
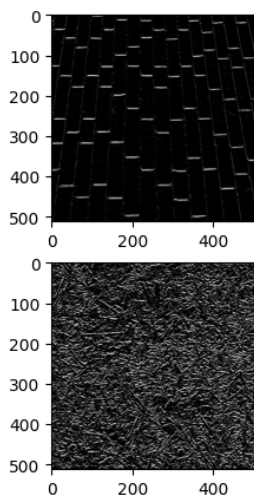
```
[ ]: filtres = [h_lines, v_lines, edges, sharper, blur]

for j,filtre in enumerate(filtres):
    fig, axes = plt.subplots(2, 2, figsize=(15, 5))
    axes = axes.ravel()

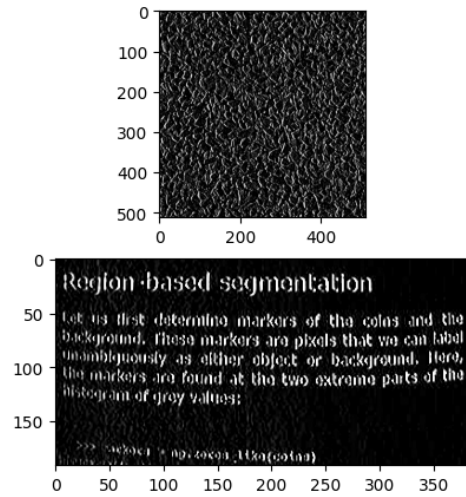
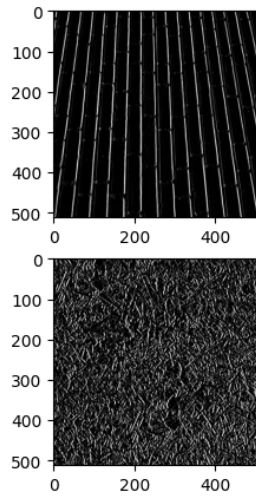
    for i, imatge in enumerate(imatges):
        resultat = cv2.filter2D(imatge, -1, filtre)
        axes[i].imshow(resultat, cmap='gray')

plt.suptitle(f'Filtre aplicat: {noms_filtres[j]}', fontsize=16)
```

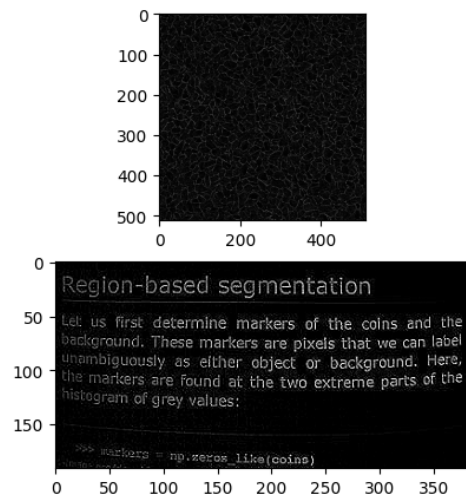
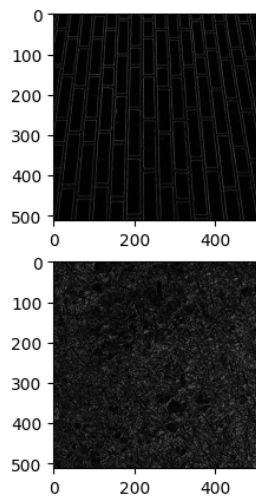
Filtre aplicat: h_lines



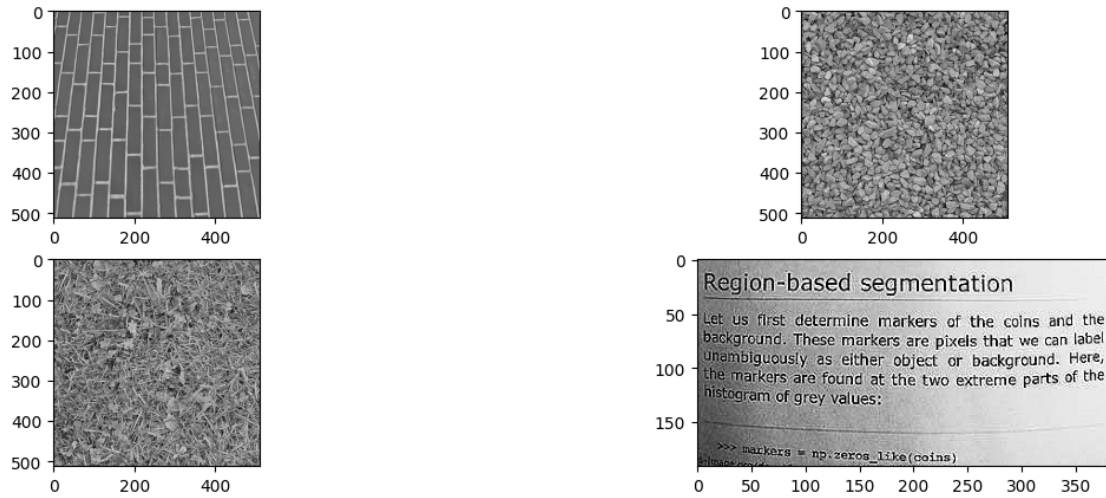
Filtre aplicat: v_lines



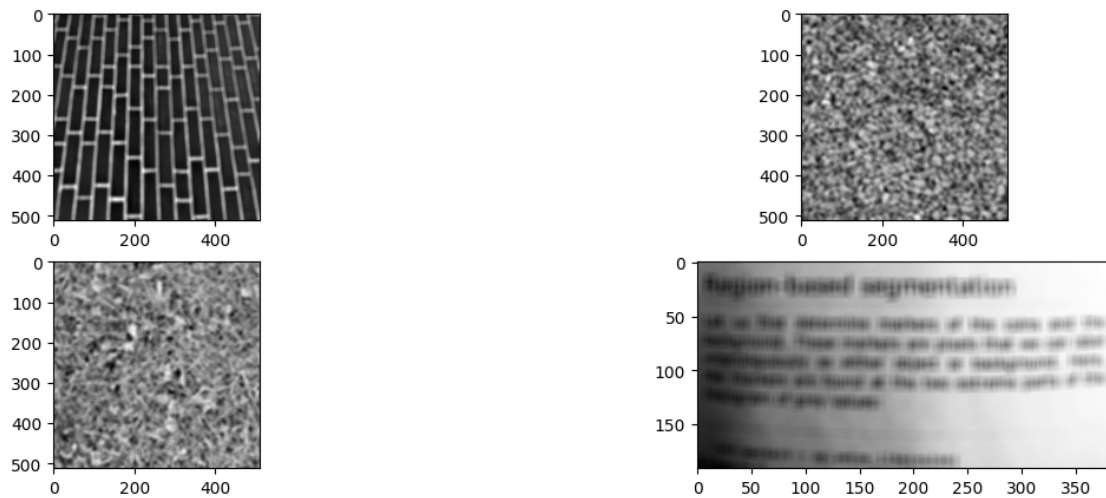
Filtre aplicat: edges



Filtre aplicat: sharper



Filtre aplicat: blur



Els filtres en processament d'imatges es divideixen principalment en dues categories: filtres de modificació i filtres de detecció/classificació.

Els primers, com el *Blur* (suavitza textures) o *Sharpen* (realça detalls), alteren l'aparença visual de la imatge per millorar la seva qualitat o estètica.

Els segons, com *Sobel/Laplacian* (detecten vores) o *Canny* (contorns), no busquen “embellir” sinó extreure informació (línies, formes, textures) per a tasques posteriors com reconeixement de màtrícules, detecció d'objectes o segmentació.

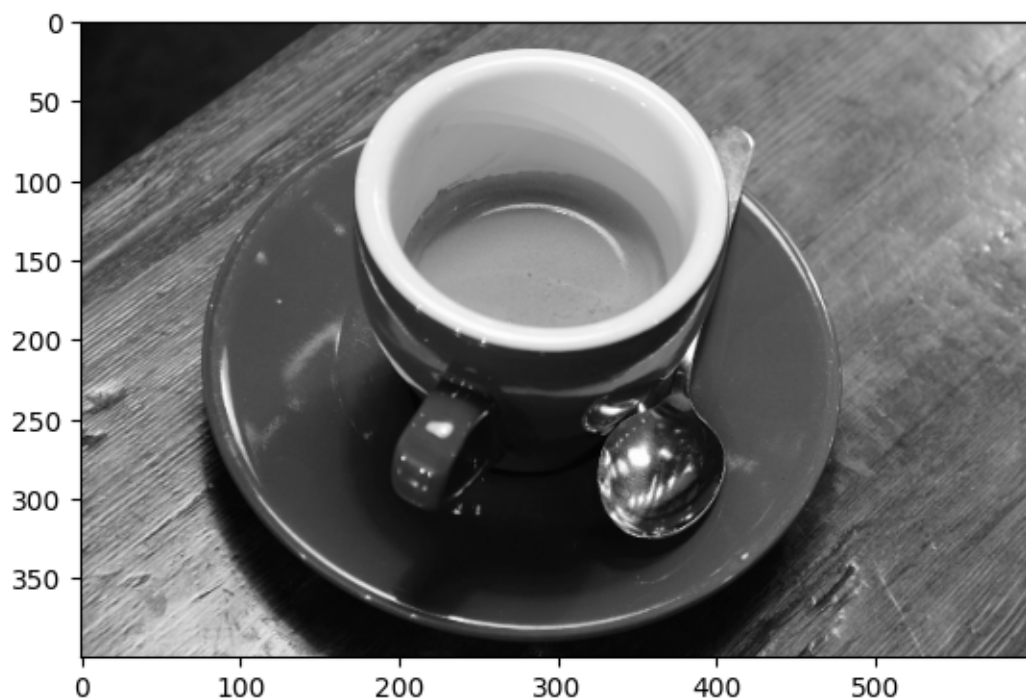
0.3 Algoritmes de detecció

Veurem un exemple de com detectar elements circulars (monedes, bombetes, etc.) que no és un exercici fàcil i no hi ha un filtre que ho detecti automàticament, sino que és el resultat d'aplicar diversos filtres en diverses etapes.

Detecció de Formes (exemple HoughCircles)

```
[ ]: img = data.coffee()
gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
output = img.copy()

plt.imshow(gray, cmap='gray')
plt.show()
```



```
[ ]: blurred = cv2.medianBlur(gray, 11) # desenfoca per reduir soroll # Elimina soroll

circles = cv2.HoughCircles(
    blurred,
    cv2.HOUGH_GRADIENT,
    dp=1,          # Resolució completa
    minDist=100,   # Distància centres
    param1=90,     # Canny high threshold
    param2=30,     # Acumulador
```

```

minRadius=80,
maxRadius=160
)

```

```

[ ]: if circles is not None:
    circles = np.round(circles[0, :]).astype("int")
    for (x, y, r) in circles:
        # Cercle verd gruix 4
        cv2.circle(output, (x, y), r, (0, 255, 0), 4)
        # Centre vermell
        cv2.circle(output, (x, y), 2, (0, 0, 255), 3)

    print(f"Detectats {len(circles)} cercles")
else:
    print("Cap cercle detectat")

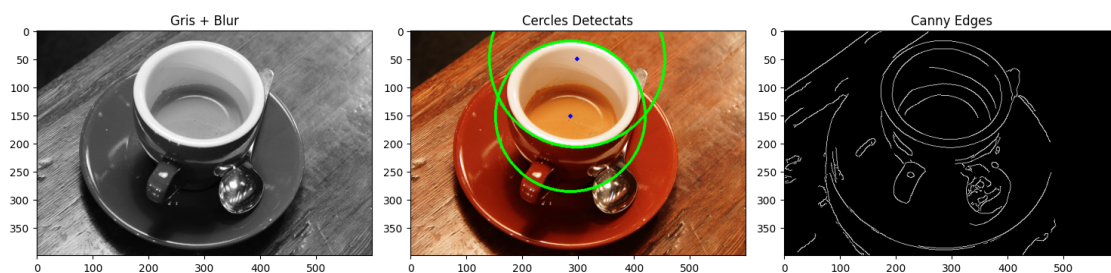
```

Detectats 2 cercles

```

[ ]: plt.figure(figsize=(15,5))
plt.subplot(131); plt.imshow(gray, cmap='gray')
plt.title('Gris + Blur')
plt.subplot(132)
#plt.imshow(cv2.cvtColor(output, cv2.COLOR_BGR2RGB), cmap = "gray")
plt.imshow(output, cmap = "gray")
plt.title('Cercles Detectats')
plt.subplot(133)
plt.imshow(cv2.Canny(blurred, 100, 35), cmap='gray')
plt.title('Canny Edges')
plt.tight_layout()
plt.show()

```



Filtres aplicats:

- Blur (Gaussian): Eliminar “soroll” de la imatge.
- Edges (Canny): Troba els contorns.

- Acumulador (Hough) : Cada punt del contorn “vota” on creu que hi ha un cercle (el centre).
- Filtres finals: Eliminar duplicats i eliminació “falsos positius”.

Els centres amb més vots i que superen els filtres son els cercles detectats.

Detecció de rostres (Haar Cascade) Desenvolupat per Viola i Jones el 2001.

Basat en patrons rectangulars que s'apliquen sobre la imatge (simplificació)

Haar Features Principals:

Cada feature és un patró rectangular amb zones negres (+1) i blanques (-1).

Calculem el resultat d'aplicació del filtre restant la suma dels píxels de les zones blanques de la suma dels píxels de les zones negres.

Si el resultat és un valor alt, significa que la imatge en aquella posició coincideix amb el patró que busquem (per exemple, ulls foscos entre front i galtes clares), i per tant hem detectat la característica.

“ Edge Vertical: Edge Horitzontal: Line Vertical:

```
[ ]: import cv2
import requests
import numpy as np
import matplotlib.pyplot as plt

url = 'https://images.unsplash.com/photo-1438761681033-6461ffad8d80?w=400'
img = cv2.imdecode(np.frombuffer(requests.get(url).content, np.uint8), cv2.
    ↳IMREAD_COLOR)

# Convertir a escala de grisos
gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
[ ]: # Carreguem els classificadors pre-entrenats
face_cascade = cv2.CascadeClassifier(cv2.data.harcascades +
    ↳'haarcascade_frontalface_default.xml')
eye_cascade = cv2.CascadeClassifier(cv2.data.harcascades + 'haarcascade_eye.
    ↳xml')
smile_cascade = cv2.CascadeClassifier(cv2.data.harcascades +
    ↳'haarcascade_smile.xml')
```

```
[ ]: # Detectar cara
faces_portrait = face_cascade.detectMultiScale(
    gray_img,
    scaleFactor=1.05,
    minNeighbors=4,
    minSize=(50, 50)
)
```



```

smiles = smile_cascade.detectMultiScale(
    gray_img,
    scaleFactor=2,
    minNeighbors=20,
    minSize=(25, 25)
)

eyes = eye_cascade.detectMultiScale(
    gray_img,
    scaleFactor=1.2,
    minNeighbors=8,
    minSize=(20, 20)
)

for (x, y, w, h) in faces_portrait:
    cv2.rectangle(img, (x, y), (x + w, y + h), (0, 255, 0), 2)

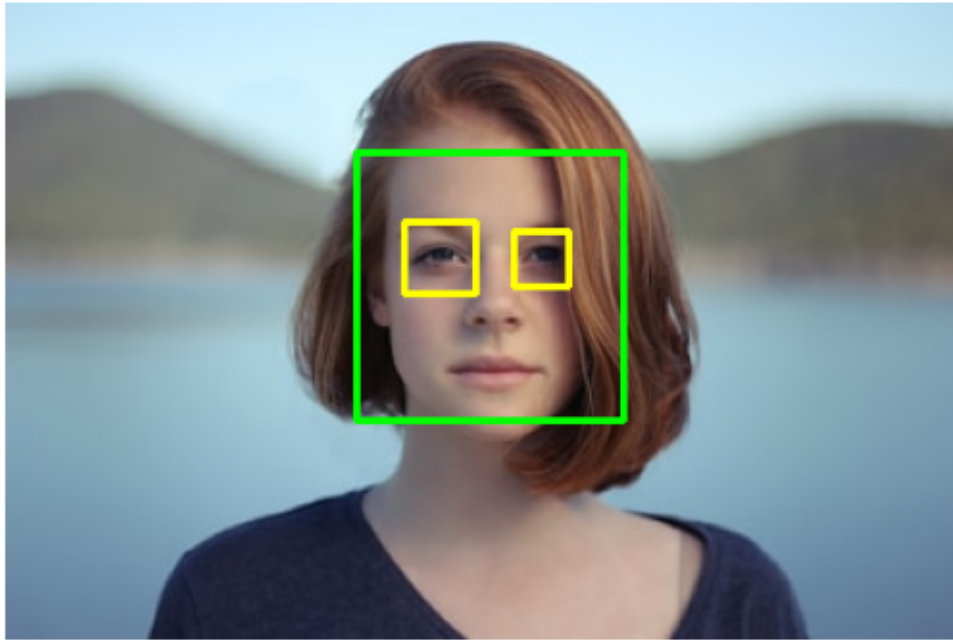
# Rectangles grocs per ulls
for (ex, ey, ew, eh) in eyes:
    cv2.rectangle(img, (ex, ey), (ex + ew, ey + eh), (0, 255, 255), 2)

# Rectangles magenta per somriures
for (sx, sy, sw, sh) in smiles:
    cv2.rectangle(img, (sx, sy), (sx + sw, sy + sh), (255, 0, 255), 2)

# Mostra la imatge
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
plt.imshow(img_rgb)
plt.title('Deteccions')
plt.axis('off')
plt.show()

```

Deteccions



VÍDEO RECOMANAT: * How Does a Computer Know You Have a Face?
https://www.youtube.com/watch?v=iht5zlGV_X4

Per que necessitem Deep Learning?

0.3.1 Limitacions del processament classic:

1. Disseny manual de característiques
2. Poca generalització
3. Complexitat creixent
4. Ajust de paràmetres difícil

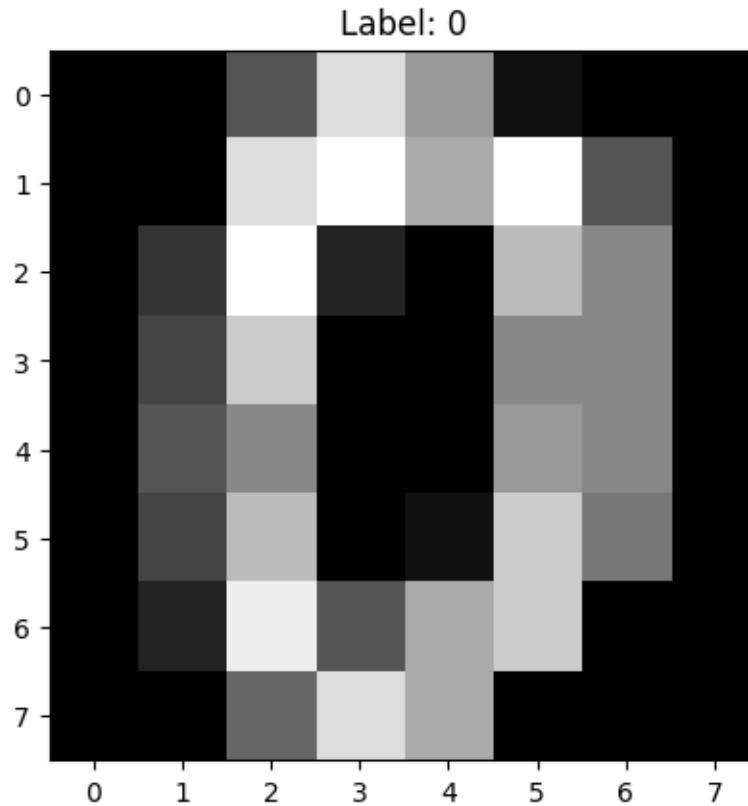
0.3.2 Exemple: Reconeixement de Digits

Compararem 3 approaches de menys a mes sofisticat: 1. **k-NN** (k-Nearest Neighbors) - ML simple
2. **SVM** (Support Vector Machine) - ML més sofisticat 3. **CNN** - Deep Learning

```
[ ]: # Carregar dataset de digits
digits = load_digits()
X, y = digits.data, digits.target
print(f'Shape de les dades: {X.shape}')

# Mostrar algun exemple
plt.imshow(digits.images[0], cmap='gray')
plt.title(f'Label: {digits.target[0]}')
plt.show()
```

Shape de les dades: (1797, 64)



0.3.3 k-NN (k-Nearest Neighbors)

Idea: Classificar segons els veïns més propers.

Per classificar una nova imatge: - Calculem la distància amb TOTES les imatges d'entrenament - Agafem les k imatges més properes - Votem: la classe més freqüent entre els k veïns

```
[ ]: # Dividir dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    random_state=42)

# Entrenar k-NN
print('Entrenant k-NN amb k=3...')
start_time = time.time()
knn_model = KNeighborsClassifier(n_neighbors=3)
knn_model.fit(X_train, y_train)
train_time_knn = time.time() - start_time

# Avaluar
print('Fent prediccions...')
```

```

start_time = time.time()
y_pred_knn = knn_model.predict(X_test)
predict_time_knn = time.time() - start_time
accuracy_knn = accuracy_score(y_test, y_pred_knn)

print(f'\nPrecisio k-NN: {accuracy_knn*100:.2f}%')
print(f'Temps d\'entrenament: {train_time_knn:.3f}s')
print(f'Temps de predicció: {predict_time_knn:.3f}s')

```

Entrenant k-NN amb k=3...

Fent prediccions...

Precisio k-NN: 98.33%

Temps d'entrenament: 0.004s

Temps de predicció: 0.011s

```

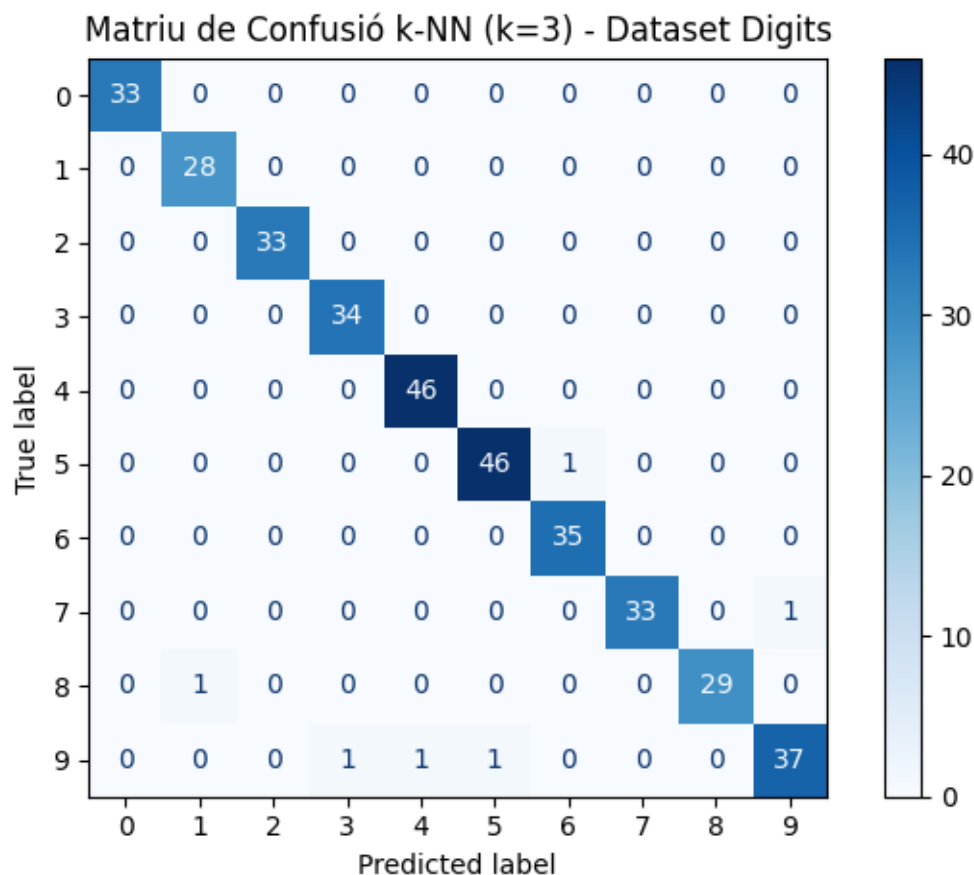
[ ]: from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred_knn)

plt.figure(figsize=(10, 8))
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=np.unique(y))
disp.plot(cmap='Blues', values_format='d')
plt.title('Matriu de Confusió k-NN (k=3) - Dataset Digits')
plt.tight_layout()
plt.show()

```

<Figure size 1000x800 with 0 Axes>



0.3.4 SVM (Support Vector Machine)

Idea: Trobar la millor “frontera” que separi les classes.

Com funciona SVM?

Objectiu: Trobar un hiperpla (línia en 2D, pla en 3D, hiperpla en +3D) que separi les classes amb el màxim marge possible entre elles.

Support Vectors: Són els punts de dades més propers a la frontera de decisió. Aquests “critics” defineixen exactament on queda el hiperpla - si es mouen, canvia la frontera.

Kernel Trick: Quan les dades NO són linealment separables en l'espai original, SVM afegeix dimensions transformant les dades a un espai de major dimensionalitat on SÍ es poden separar amb un hiperpla lineal.

```
[ ]: start_time = time.time()
      svm_model = SVC(kernel='rbf', gamma='scale') # kernel no lineal més utilitzat
      svm_model.fit(X_train, y_train)
      train_time_svm = time.time() - start_time
```

```

start_time = time.time()
y_pred_svm = svm_model.predict(X_test)
predict_time_svm = time.time() - start_time
accuracy_svm = accuracy_score(y_test, y_pred_svm)

print(f'\nPrecisio SVM: {accuracy_svm*100:.2f}%')
print(f'Temps d\'entrenament: {train_time_svm:.3f}s')
print(f'Temps de predicció: {predict_time_svm:.3f}s')

print(f'\nCOMPARACIO k-NN vs SVM:')
print(f"{'Metode':<15} {'Precisio':<12} {'Temps Train':<15} {'Temps Predict':<15}")
print(f"{'k-NN':<15} {accuracy_knn*100:>6.2f}%      {train_time_knn:>10.3f}s      ↳  
↳ {predict_time_knn:>10.3f}s")
print(f"{'SVM':<15} {accuracy_svm*100:>6.2f}%      {train_time_svm:>10.3f}s      ↳  
↳ {predict_time_svm:>10.3f}s")

```

Precisio SVM: 98.61%
 Temps d'entrenament: 0.049s
 Temps de predicció: 0.021s

COMPARACIO k-NN vs SVM:

Metode	Precisio	Temps Train	Temps Predict
k-NN	98.33%	0.004s	0.011s
SVM	98.61%	0.049s	0.021s

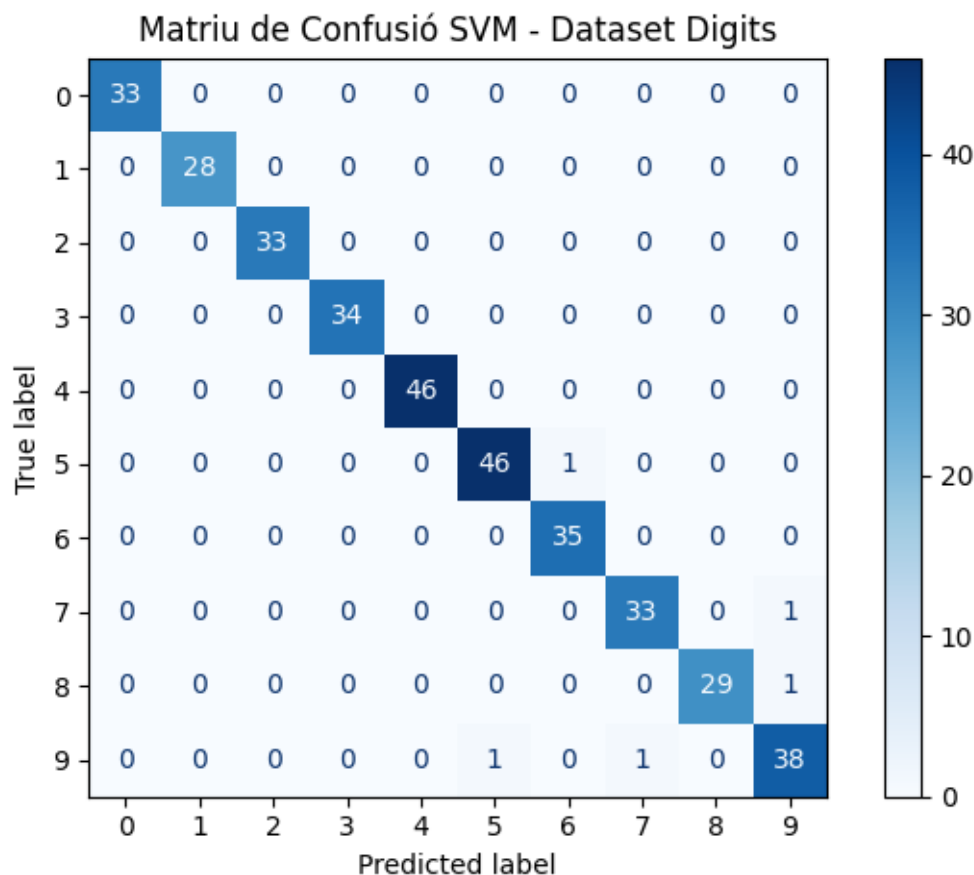
```

[ ]: cm = confusion_matrix(y_test, y_pred_svm)

plt.figure(figsize=(10, 8))
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=np.unique(y))
disp.plot(cmap='Blues', values_format='d')
plt.title('Matriu de Confusió SVM - Dataset Digits')
plt.tight_layout()
plt.show()

```

<Figure size 1000x800 with 0 Axes>



0.3.5 Deep Learning: CNN

Ara veurem com les **CNNs aprenen automàticament** les millors característiques.

```
[ ]: X_cnn = digits.images.reshape(-1, 8, 8, 1).astype('float32') / 16.0
y_cnn = keras.utils.to_categorical(y, 10)
X_train_cnn, X_test_cnn, y_train_cnn, y_test_cnn = train_test_split(
    X_cnn, y_cnn, test_size=0.2, random_state=42
)

print(f'Shape dades per CNN: {X_train_cnn.shape}')
print(f'Shape labels (one-shot): {y_train_cnn.shape}')
```

Shape dades per CNN: (1437, 8, 8, 1)

Shape labels (one-shot): (1437, 10)

```
[ ]: # Construïm una CNN simple
model_cnn = keras.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(8,8,1)),
    layers.Conv2D(64, (3,3), activation='relu'),
```

```

        layers.MaxPooling2D(2,2),
        layers.Dropout(0.25),
        layers.Flatten(),
        layers.Dense(128, activation='relu'),
        layers.Dropout(0.5),
        layers.Dense(10, activation='softmax')
    ])

model_cnn.compile(optimizer='adam', loss='categorical_crossentropy',
    metrics=['accuracy'])
model_cnn.summary()

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
conv2d_4 (Conv2D)	(None, 6, 6, 32)	320
conv2d_5 (Conv2D)	(None, 4, 4, 64)	18,496
max_pooling2d_2 (MaxPooling2D)	(None, 2, 2, 64)	0
dropout_4 (Dropout)	(None, 2, 2, 64)	0
flatten_2 (Flatten)	(None, 256)	0
dense_4 (Dense)	(None, 128)	32,896
dropout_5 (Dropout)	(None, 128)	0
dense_5 (Dense)	(None, 10)	1,290

Total params: 53,002 (207.04 KB)

Trainable params: 53,002 (207.04 KB)

Non-trainable params: 0 (0.00 B)


```
[ ]: start_time = time.time()
history = model_cnn.fit(X_train_cnn, y_train_cnn, validation_split=0.2,
                        epochs=20, batch_size=32, verbose=0)
train_time_cnn = time.time() - start_time

# Avaluar
test_loss, test_accuracy = model_cnn.evaluate(X_test_cnn, y_test_cnn, verbose=0)

print(f"'k-NN (simple)':<20> {accuracy_knn*100:>6.2f}%")
print(f"'SVM (classic)':<20> {accuracy_svm*100:>6.2f}%")
print(f"'CNN (deep learning)':<20> {test_accuracy*100:>6.2f}%")
```

```
k-NN (simple)          98.33%
SVM (classic)         98.61%
CNN (deep learning)   98.89%
```

```
[ ]: start_time = time.time()
y_pred_cnn = model_cnn.predict(X_test_cnn)
predict_time_cnn = time.time() - start_time

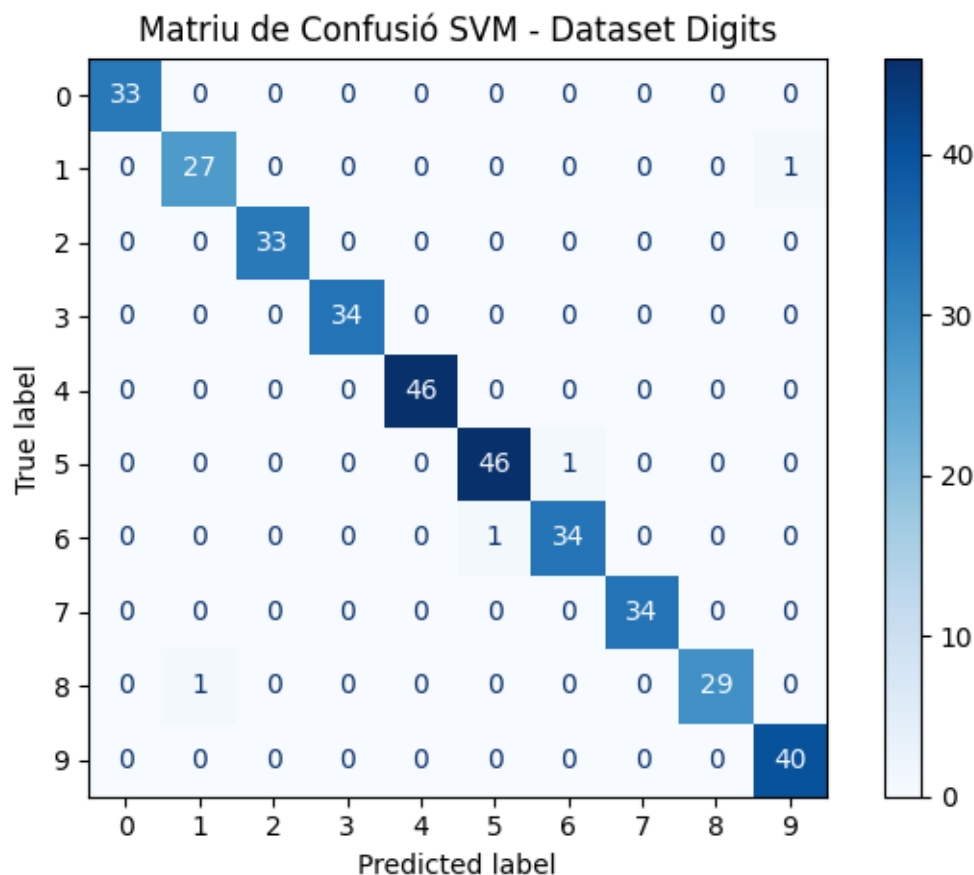
y_pred_classes = np.argmax(y_pred_cnn, axis=1)
y_test_classes = np.argmax(y_test, axis=1) if y_test.ndim > 1 else y_test

cm = confusion_matrix(y_test_classes, y_pred_classes)

plt.figure(figsize=(10, 8))
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=np.unique(y))
disp.plot(cmap='Blues', values_format='d')
plt.title('Matriu de Confusió SVM - Dataset Digits')
plt.tight_layout()
plt.show()
```

```
12/12          0s 8ms/step
```

```
<Figure size 1000x800 with 0 Axes>
```



```
[ ]: print(f'\nCOMPARACIO k-NN vs SVM vs CNN:')
print(f"{'Metode':<15} {'Precisio':<12} {'Temps Train':<15} {'Temps Predict':<15}")
print(f"{'k-NN':<15} {accuracy_knn*100:>6.2f}%      {train_time_knn:>10.3f}s      ↳
↳ {predict_time_knn:>10.3f}s")
print(f"{'SVM':<15} {accuracy_svm*100:>6.2f}%      {train_time_svm:>10.3f}s      ↳
↳ {predict_time_svm:>10.3f}s")
print(f"{'CNN':<15} {test_accuracy*100:>6.2f}%      {train_time_cnn:>10.3f}s      ↳
↳ {predict_time_cnn:>10.3f}s")
```

COMPARACIO k-NN vs SVM vs CNN:

Metode	Precisio	Temps Train	Temps Predict
k-NN	98.33%	0.004s	0.011s
SVM	98.61%	0.049s	0.021s
CNN	98.89%	8.408s	0.222s

0.4 Xarxes Neuronals Convolucionals (CNNs)

0.4.1 Components principals:

1. **Capes Convolucionals:** Detecten característiques locals
2. **Pooling:** Redueixen dimensionalitat i creen invariancia
3. **Capes Denses:** Combinen característiques per classificar
 - Aprenen jerarquies: vores -> formes -> objectes
 - Detecten patrons independentment de la posició

0.4.2 Que detecten les Hidden Layers?

Les CNNs aprenen una **jerarquia de característiques** que va de simples a complexes.

Jerarquia d'aprenentatge en una CNN:

- **CAPA 1 (Conv2D - 32 filtres):** Detecta VORES I COLORS BASICS
 - Vores horitzontals, verticals, diagonals
 - Colors basics, gradients
- **CAPA 2-3 (Conv2D - 64 filtres):** Detecta FORMES I TEXTURES
 - Cercles, quadrats, cantonades, arcs
 - Textures (pell, llisa, rugosa)
- **CAPA 4-5 (Conv2D - 128 filtres):** Detecta PARTS D'OBJECTES
 - Ulls, orelles, nas, dents, potes (en animals)
 - Rodes, finestres, portes (en objectes)
- **CAPA 6+ (Dense layers):** Detecta OBJECTES COMPLETS I CONCEPTES
 - Gat, gos, cotxe, casa
 - Combinació de totes les parts

Exemple: Detectar un COTXE complet

CAPA 1 (32 filtres simples): - Filtre 1: Vora horitzontal (capo) - Filtre 5: Vora vertical (lateral) - Filtre 8: Diagonal (parabrisa)

CAPA 2 (64 filtres que combinen resultats de capa 1): - Filtre 15: Cercle (roda) - Filtre 23: Rectangle (finestra)

CAPA 3 (128 filtres): - Filtre 34: Part frontal del cotxe - Filtre 89: Vista lateral completa

CAPA FINAL: - Combina tot: "Te rodes + finestres + forma allargada = COTXE"

##RESUM

El processament d'imatges clàssic (o tradicional) utilitza regles manuals i operacions matemàtiques fixes, com filtres o detecció de vores, però té limitacions en escenaris complexos perquè no aprèn patrons automàticament.

0.4.3 Processament Clàssic:

Depèn de regles programades (com OpenCV per binarització o Canny per vores) i funciona bé en condicions ideals, però falla amb variacions com il·luminació irregular, oclusió o deformacions.

Quines tasques pot realitzar:

Detecció simple, com línies rectes, cercles o taques en fons uniformes (ex: comptar monedes en una foto plana). No pot classificar objectes diversos sense reescriure codi per cada cas.

###Machine Learning (ML): Aquí extreus característiques manuals (com color, forma, HOG) i entrenes classificadors com SVM o Random Forest.

Quines tasques pot realitzar:

Detecció complexa i classificació, com cares bàsiques (Haar cascades), cotxes en vistes estàndards o defectes industrials específics.

###Deep Learning: Les xarxes neuronals convolucionals (CNN) aprenen característiques directament dels píxels. És més en robustes davant variacions (rotacions, escales, soroll).

Quines tasques pot realitzar: Segmentació (U-Net per píxel a píxel, ex: separar tumors en imatges mèdiques), detecció d'objectes (YOLO per múltiples objectes en temps real, com cotxes i vianants), reconeixement de cares (FaceNet) o estimació de posició corporal (OpenPose per detectar postures humanes en vídeo).

VÍDEO RECOMANAT:

- “Neural Network Simply Explained”: <https://www.youtube.com/watch?v=i1AqHG4k8mE>
- “Redes Neuronales CONVOLUCIONALES”: <https://www.youtube.com/watch?v=V8j1oENVz00>